Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 17

Image Generation

Two ways to build a latent space of images — one by construction, one by denoising in a loop — and a walk between two prompts that shows what a neural network really is.

Duration 3.5 hours (3 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 17

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Explain what a **latent space of images** is, and why sampling from one produces images nobody has drawn.
2. Describe a **classical autoencoder** and say why its latent space is not useful for generation.
3. Build a **VAE**: encoder to a mean and variance, a sampling layer, a decoder — and both halves of its loss.
4. Customise training with **compute_loss()** rather than train_step(), and say why that keeps the code backend-agnostic.
5. Explain **reverse diffusion** as a denoising autoencoder in a loop, and define diffusion time and the diffusion schedule.
6. Build a **U-Net denoiser** that predicts a noise mask, and train it on the Oxford Flowers dataset.
7. Use a pretrained **Stable Diffusion** text-to-image model, with negative prompts and a controllable step count.
8. Interpolate between two prompts with **slerp**, and explain why spherical interpolation beats linear.

BOOK
Chapter 17 — full text

NOTEBOOK
01_vae_on_mnist.ipynb

NOTEBOOK
02_vae_latent_space_grid.ipynb

NOTEBOOK
03_diffusion_unet_and_schedule.ipynb

NOTEBOOK
04_training_the_flower_diffuser.ipynb

NOTEBOOK
05_stable_diffusion_latent_walk.ipynb

NOTEBOOK
All chapter 17 notebooks

REPO
Author's official notebook

01

Latent spaces of images

The one idea underneath every image generator.

SECTION 17.1.1

A space where every point is a valid image

The key idea of image generation: develop a **low-dimensional latent space** — a vector space, like everything else in deep learning — in which any point maps to an image that looks like the real thing.

The module that realises the mapping, taking a latent point and outputting a grid of pixels, is called a **generator**, or sometimes a **decoder**.

. . . and a walk between two points in it

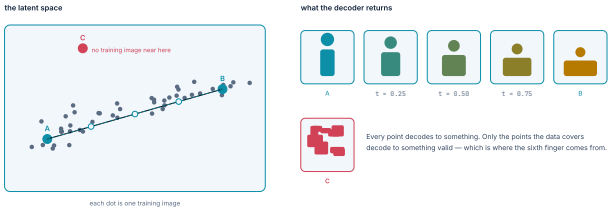


Figure 17.1 — the space, two points in it, and what the decoder returns along the line between them. Step through it; the last step leaves the region the data covers.

The **in-betweens of the training images** — a claim about **geometry**, not understanding. Walk the line between two of them and every point on it decodes to an image nobody ever took.

SECTION 17.1.1

Add text conditioning and you get a brush

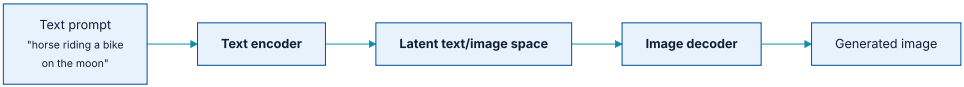


Figure 17.2 — a prompt lands somewhere in the joint space, and the decoder renders it.

Interpolating between many training images lets such models generate **infinite combinations of visual concepts**, including many nobody had explicitly come up with. A horse riding a bike on the moon? You got it.

The latent space does not model the physical world

> Despite having seen tens of thousands of images of people riding bikes, the model does not understand in a human sense what riding a bike *means* — pedalling, steering, balance. Which is why your horse is **unlikely to be depicted pedalling with its hind legs** the way a human artist would draw it.

This is chapter 15's interpolation argument, arriving in a different modality. **Nothing about pixels changes it.**

Three families, and why one is missing

Diffusion models

The architecture behind **nearly all commercial image generation services today**. Section 17.2.

Variational autoencoders

Highly structured, controllable latent spaces. Not the first choice for fidelity, but still in use. Section 17.1.

GANs

Covered in previous editions of this book. They have **gradually fallen out of fashion** and been all but replaced by diffusion.

Note also that none of this is image-specific. You could develop latent spaces of **sound or music** with the same models — pictures are simply where the most interesting results have been obtained so far.

02

Variational autoencoders

A little statistical magic that forces a latent space to be continuous.

Start with the classical autoencoder

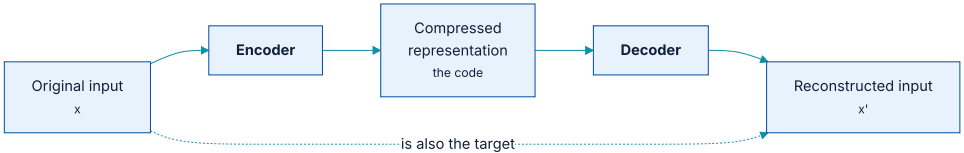


Figure 17.3 — the target is the input. This is self-supervised learning.

By constraining the **code** — most commonly to be low dimensional and sparse — the encoder becomes a way of compressing the input into fewer bits.

SECTION 17.1.2

And note that classical autoencoders do not work well

In practice they lead to latent spaces that are neither useful nor nicely structured. **They are not much good at compression either.** For these reasons they have largely fallen out of fashion.

VAEs augment the autoencoder with a little statistical magic that forces it to learn **continuous, highly structured** latent spaces — and that turns out to be a powerful tool for image generation.

VAEs were discovered simultaneously by **Kingma and Welling** in December 2013 and by **Rezende, Mohamed, and Wierstra** in January 2014. They mix deep learning with Bayesian inference, and remain in use in research a decade later.

Encode to a distribution, not to a point

The assumption is that the image was generated by a statistical process, and that the randomness of that process should be accounted for during encoding and decoding.

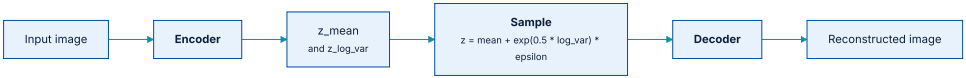


Figure 17.4 — the encoder outputs two vectors; a point is sampled from the distribution they define; the decoder reconstructs from that point.

The mechanism, stated precisely

1. An **encoder** turns `input_img` into two parameters in the latent space, `z_mean` and `z_log_variance`.
2. A point is **randomly sampled** from the latent normal distribution assumed to have generated the image: `z = z_mean + exp(z_log_variance) * epsilon`, where `epsilon` is a random tensor of small values.
3. A **decoder** maps that point back to the original input image.

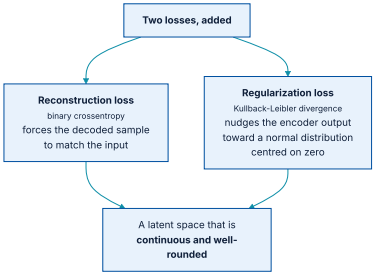
> Because `epsilon` is random, **every point close to where `input_img` was encoded decodes to something similar to `input_img`.** That is what forces the latent space to be **continuously meaningful**.

Continuity plus low dimensionality gives you concept vectors

The result is a very structured space, highly suitable to manipulation via **concept vectors** — the same *word arithmetic* idea from chapter 15, now over faces and objects rather than words.

This is why VAEs remain relevant even where diffusion produces better images: when **interpretability, control over the latent space, and reconstruction** matter, they are still the right tool.

Two losses, pulling in different directions



Reconstruction alone would collapse to a classical autoencoder; regularization alone would ignore the data.

The regularization term is the **Kullback-Leibler divergence**, which nudges the distribution of the encoder output toward a well-rounded normal distribution centred on 0 — a sensible prior about the structure of the space being modelled.

The encoder: a ConvNet with two heads

listing 17.1 PYTHON

```python
import keras
from keras import layers

latent_dim = 2

image_inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(
    image_inputs
)
x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Flatten()(x)
x = layers.Dense(16, activation="relu")(x)
z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
encoder = keras.Model(image_inputs, [z_mean, z_log_var], name="encoder")
```

Strides are preferable to max pooling for **any model that cares where things are in the image** — as chapter 11's segmentation example did, and as this one does, since the encoding must support reconstructing a valid image.

Sixty-nine thousand parameters, and a two-dimensional output

OUTPUT

```
>>> encoder.summary()
Model: "encoder"
| input_layer (InputLayer) | (None, 28, 28, 1)  |      0 |
| conv2d (Conv2D)          | (None, 14, 14, 32) |    320 |
| conv2d_1 (Conv2D)        | (None, 7, 7, 64)   | 18,496 |
| flatten (Flatten)        | (None, 3136)       |      0 |
| dense (Dense)            | (None, 16)         | 50,192 |
| z_mean (Dense)           | (None, 2)          |     34 |
| z_log_var (Dense)        | (None, 2)          |     34 |
 Total params: 69,076 (269.83 KB)
```

The whole of MNIST is being pushed through a **two-number** bottleneck. That is deliberate: a 2D latent space is one we can plot in full, which is the point of this example.

The sampling layer

listing 17.2 PYTHON

```python
from keras import ops

class Sampler(keras.Layer):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.seed_generator = keras.random.SeedGenerator()
        self.built = True

    def call(self, z_mean, z_log_var):
        batch_size = ops.shape(z_mean)[0]
        z_size = ops.shape(z_mean)[1]
        epsilon = keras.random.normal(
            (batch_size, z_size), seed=self.seed_generator
        )
        return z_mean + ops.exp(0.5 * z_log_var) * epsilon
```

`exp(0.5 * z_log_var)` converts a **log variance** to a standard deviation. Predicting the log rather than the variance itself keeps the encoder's output unconstrained — it may be negative, and the exponential makes it positive.

The decoder mirrors the encoder

listing 17.3 PYTHON

```python
latent_inputs = keras.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Conv2DTranspose(32, 3, activation="relu", strides=2, padding="same")(x)
decoder_outputs = layers.Conv2D(1, 3, activation="sigmoid", padding="same")(x)
decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")
```

The `Dense(7 * 7 * 64)` produces exactly the number of coefficients the encoder's `Flatten` consumed — the architecture is a **mirror**, and reading it that way is the quickest way to check it.

compute_loss() instead of train_step()

But `train_step()` contents are **backend specific** — `GradientTape` under TensorFlow, `loss.backward()` under PyTorch. Overriding `compute_loss()` instead keeps the default `train_step()` and stays **backend agnostic**.

the signature

PYTHON

```
compute_loss(x, y, y_pred, sample_weight=None, training=True)
```

`x` is the input, `y` the target — **None** here, since our dataset has no targets — and `y_pred` is the output of `call()`. The method returns a scalar, and may also update metric state.

The VAE model: constructor

listing 17.4 - constructor PYTHON

```python
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.sampler = Sampler()
        self.reconstruction_loss_tracker = keras.metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    def call(self, inputs):
        return self.encoder(inputs)
```

Note that `call()` returns only the **encoder output** — the two latent parameters. Sampling and decoding happen inside the loss, because that is where the reconstruction is needed.

The VAE model: the loss

listing 17.4 - compute_loss PYTHON

```python
def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
    original = x
    z_mean, z_log_var = y_pred
    reconstruction = self.decoder(self.sampler(z_mean, z_log_var))
    reconstruction_loss = ops.mean(
        ops.sum(
            keras.losses.binary_crossentropy(x, reconstruction), axis=(1, 2)
        )
    )
    kl_loss = -0.5 * (
        1 + z_log_var - ops.square(z_mean) - ops.exp(z_log_var)
    )
    total_loss = reconstruction_loss + ops.mean(kl_loss)
    self.reconstruction_loss_tracker.update_state(reconstruction_loss)
    self.kl_loss_tracker.update_state(kl_loss)
    return total_loss
```

The reconstruction loss is **summed over the spatial dimensions** (axes 1 and 2) and averaged over the batch. The KL term is the closed-form divergence between the encoder's distribution and a unit normal.

Training with no loss and no targets

listing 17.5 PYTHON

```python
import numpy as np

(x_train, _), (x_test, _) = keras.datasets.mnist.load_data()
mnist_digits = np.concatenate([x_train, x_test], axis=0)
mnist_digits = np.expand_dims(mnist_digits, -1).astype("float32") / 255

vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())
vae.fit(mnist_digits, epochs=30, batch_size=128)
```

We concatenate the training and test splits: there is no test set to protect here, because there is **nothing being evaluated against held-out labels**. We want the latent space to see every digit there is.

Walking the latent space on a grid

listing 17.6 PYTHON

```python
import matplotlib.pyplot as plt

n = 30
digit_size = 28
figure = np.zeros((digit_size * n, digit_size * n))
grid_x = np.linspace(-1, 1, n)
grid_y = np.linspace(-1, 1, n)[::-1]

for i, yi in enumerate(grid_y):
    for j, xi in enumerate(grid_x):
        z_sample = np.array([[xi, yi]])
        x_decoded = vae.decoder.predict(z_sample)
        digit = x_decoded[0].reshape(digit_size, digit_size)
        figure[
            i * digit_size : (i + 1) * digit_size,
            j * digit_size : (j + 1) * digit_size,
        ] = digit

plt.imshow(figure, cmap="Greys_r")
```

Nine hundred digits, none of which are in MNIST.

What the grid shows

A **completely continuous** distribution of the different digit classes, with one digit morphing into another as you follow a path through the latent space.

Specific directions in the space have meaning: there is a direction for *four-ness*, one for *one-ness*, and so on. Nobody labelled them. They fall out of the continuity constraint.

> This is the payoff for the whole VAE construction. A classical autoencoder's latent space would show **islands of digits separated by regions that decode to nothing** — the KL term is what fills the gaps.

03

Diffusion models

A denoising autoencoder in a loop — and the architecture behind nearly every commercial image generator today.

SECTION 17.2

Denoising is the one task autoencoders excel at

In the late 2010s this gave rise to successful **image super-resolution** models, which have shipped in every major smartphone camera app for years.

> These models are **not** recovering lost detail hidden in the input, like the *enhance* scene in *Blade Runner*. They are making educated guesses — **hallucinating** a higher-resolution version of what you gave them.

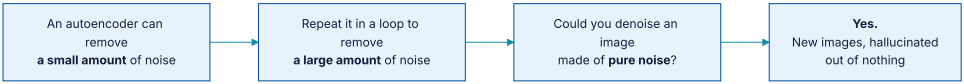
The moon that was not there

A great deal of detail that simply was not present in the printout gets straight-up hallucinated, because the super-resolution model is **overfitted to moon photography**.

> So, unlike Rick Deckard, **definitely do not use this technique for forensics**.

Worth keeping for any professional audience: an enhancement model's output is **evidence of the model's priors**, not evidence about the scene.

If a little noise, why not all of it?



The arresting idea that early denoising successes led researchers to.

These should more accurately be called **reverse diffusion** models — *diffusion* refers to the forward process of gradually adding noise to an image until it disperses into nothing.

Reverse diffusion, one step at a time

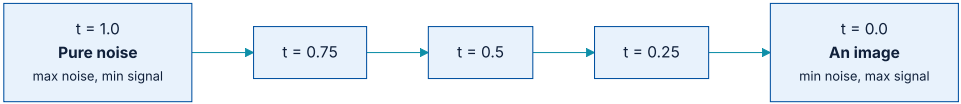


Figure 17.7 — turning pure noise into an image via repeated denoising.

> *"Every block of stone has a statue inside it and it is the task of the sculptor to discover it." — Well,* ***every square of white noise has an image inside it, and it is the task of the diffusion model to discover it.***
>
> Michelangelo, and section 17.2

Oxford Flowers: 8,189 images, 102 species

PYTHON

```python
import os

fpath = keras.utils.get_file(
    origin="https://www.robots.ox.ac.uk/~vgg/data/flowers/102/102flowers.tgz",
    extract=True,
)

batch_size = 32
image_size = 128
images_dir = os.path.join(fpath, "jpg")

dataset = keras.utils.image_dataset_from_directory(
    images_dir,
    labels=None,
    image_size=(image_size, image_size),
    crop_to_aspect_ratio=True,
)
dataset = dataset.rebatch(batch_size, drop_remainder=True)
```

Two arguments matter. `labels=None` — we want images only. `crop_to_aspect_ratio=True` — resizing without it would **distort** the images, and distortion in the training set becomes distortion in everything generated.

The denoiser predicts the noise, not the image

> And rather than outputting a denoised image, the model outputs a **predicted noise mask**, which we subtract from the input. **Predicting what to remove is an easier target than predicting what remains.**

For the architecture we use a **U-Net** — a ConvNet originally developed for image segmentation, and last seen in chapter 11.

Three stages, with skip connections across

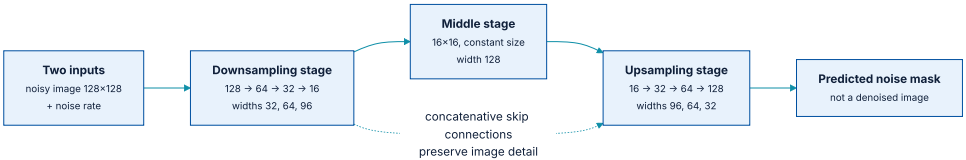


Figure 17.9 — a 1:1 mapping between downsampling and upsampling blocks.

Each upsampling block is the inverse of a downsampling block, and **concatenative residual connections** run from each downsampling block to its partner. These avoid the loss of image detail across successive down- and upsampling.

SECTION 17.2.2

The residual block

PYTHON

```python
def residual_block(x, width):
    input_width = x.shape[3]
    if input_width == width:
        residual = x
    else:
        residual = layers.Conv2D(width, 1)(x)
    x = layers.BatchNormalization(center=False, scale=False)(x)
    x = layers.Conv2D(width, 3, padding="same", activation="swish")(x)
    x = layers.Conv2D(width, 3, padding="same")(x)
    x = x + residual
    return x
```

This is the pattern from chapter 9, unchanged — with `swish` in place of `relu`, and normalization that does **not** learn a scale or centre, since the residual path carries those.

SECTION 17.2.2

The U-Net: two inputs, and the way down

get_model - inputs and downsampling PYTHON

```python
def get_model(image_size, widths, block_depth):
    noisy_images = keras.Input(shape=(image_size, image_size, 3))
    noise_rates = keras.Input(shape=(1, 1, 1))

    x = layers.Conv2D(widths[0], 1)(noisy_images)
    n = layers.UpSampling2D(image_size, interpolation="nearest")(noise_rates)
    x = layers.Concatenate()([x, n])

    skips = []
    for width in widths[:-1]:
        for _ in range(block_depth):
            x = residual_block(x, width)
            skips.append(x)
        x = layers.AveragePooling2D(pool_size=2)(x)
```

The scalar noise rate is **upsampled to the full image size** and concatenated as an extra channel — the standard way to feed a scalar condition to a convolutional network.

SECTION 17.2.2

The U-Net: the middle, and the way back up

get_model - middle and upsampling PYTHON

```python
    for _ in range(block_depth):
        x = residual_block(x, widths[-1])

    for width in reversed(widths[:-1]):
        x = layers.UpSampling2D(size=2, interpolation="bilinear")(x)
        for _ in range(block_depth):
            x = layers.Concatenate()([x, skips.pop()])
            x = residual_block(x, width)

    pred_noise_masks = layers.Conv2D(3, 1, kernel_initializer="zeros")(x)
    return keras.Model([noisy_images, noise_rates], pred_noise_masks)
```

`skips.pop()` pairs each upsampling block with its mirror on the way down — **last in, first out** is exactly the pairing the architecture diagram shows.

Zero initialization, and widening as you downsample

kernel_initializer="zeros"

The last layer predicts **only zeros** after initialization — so the model's default assumption before training is *no noise*. A deliberately chosen, harmless starting point.

widths=[32, 64, 96, 128]

Layers get **wider as the feature map shrinks**, and narrower again as it grows back. The same trade as every ConvNet in chapter 9.

You would instantiate with `get_model(image_size=128, widths=[32, 64, 96, 128], block_depth=2)`.

Diffusion time and the diffusion schedule

Diffusion time

The index of the current step, here a **continuous value between 1 and 0**. 1 is maximal noise and minimal signal; 0 is almost all signal and no noise.

Diffusion schedule

The relationship between the current time and how much noise and signal are present. We use a **cosine schedule**.

The cosine choice is not arbitrary: it maintains the identity **noise_rates^2 + signal_rates^2** 1==, so total energy stays constant as the mix shifts from one to the other.

The schedule, in seven lines

listing 17.7 PYTHON

```python
def diffusion_schedule(
    diffusion_times,
    min_signal_rate=0.02,
    max_signal_rate=0.95,
):
    start_angle = ops.cast(ops.arccos(max_signal_rate), "float32")
    end_angle = ops.cast(ops.arccos(min_signal_rate), "float32")
    diffusion_angles = start_angle + diffusion_times * (end_angle - start_angle)
    signal_rates = ops.cos(diffusion_angles)
    noise_rates = ops.sin(diffusion_angles)
    return noise_rates, signal_rates
```

The two bounds keep the process away from its extremes: never quite pure signal (0.95), never quite pure noise (0.02). Both endpoints are numerically awkward, and neither is needed.

SECTION 17.2.4

What the DiffusionModel needs to hold

- **A loss function** — mean absolute error, that is `mean(abs(real_noise_mask - predicted_noise_mask))`.
- **An image normalization layer** — the noise we add has unit variance and zero mean, so the images must be normalized the same way for their value ranges to match.

PYTHON

```python
class DiffusionModel(keras.Model):
    def __init__(self, image_size, widths, block_depth, **kwargs):
        super().__init__(**kwargs)
        self.image_size = image_size
        self.denoising_model = get_model(image_size, widths, block_depth)
        self.seed_generator = keras.random.SeedGenerator()
        self.loss = keras.losses.MeanAbsoluteError()
        self.normalizer = keras.layers.Normalization()
```

Mean absolute error rather than mean squared error: the noise mask is normally distributed, and **squared error would let a few extreme pixels dominate** the gradient.

SECTION 17.2.4

The denoise method

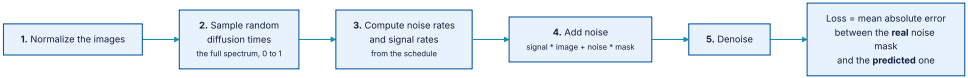
PYTHON

```
def denoise(self, noisy_images, noise_rates, signal_rates):
    pred_noise_masks = self.denoising_model([noisy_images, noise_rates])
    pred_images = (
        noisy_images - noise_rates * pred_noise_masks
    ) / signal_rates
    return pred_images, pred_noise_masks
```

Read the arithmetic as the inverse of the mixing formula: `noisy = signal * image + noise * mask`, so `image = (noisy − noise * mask) / signal`. **Two lines that undo each other exactly.**

SECTION 17.2.4

The training step, in five operations



Random diffusion times are essential: the model must be trained across the full spectrum, because it will be called at every point of it.

`call()` returns three things — the predicted images, the predicted noise masks, and the **actual** noise masks it applied. The last two are what `compute_loss()` compares.

SECTION 17.2.4

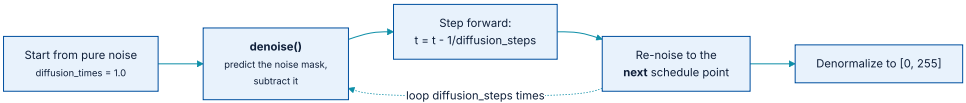
call() and compute_loss(), written out

PYTHON

```python
def call(self, images):
    images = self.normalizer(images)
    noise_masks = keras.random.normal(
        (batch_size, self.image_size, self.image_size, 3),
        seed=self.seed_generator,
    )
    diffusion_times = keras.random.uniform(
        (batch_size, 1, 1, 1), minval=0.0, maxval=1.0,
        seed=self.seed_generator,
    )
    noise_rates, signal_rates = diffusion_schedule(diffusion_times)
    noisy_images = signal_rates * images + noise_rates * noise_masks
    pred_images, pred_noise_masks = self.denoise(
        noisy_images, noise_rates, signal_rates
    )
    return pred_images, pred_noise_masks, noise_masks

def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
    _, pred_noise_masks, noise_masks = y_pred
    return self.loss(noise_masks, pred_noise_masks)
```

Note how small the loss is. **All the difficulty is in the forward pass**, and the objective itself is one comparison.

Generation: start from noise and walk the schedule down



Each iteration denoises fully, then re-noises to the next schedule point — a smaller step than the one just undone.

That re-noising is the part that surprises people. The model predicts the **whole** clean image at every step; we then deliberately add back the amount of noise appropriate to the next time index, and ask again.

SECTION 17.2.5

The generation loop: one step

generate - the loop PYTHON

```python
    def generate(self, num_images, diffusion_steps):
        noisy_images = keras.random.normal(
            (num_images, self.image_size, self.image_size, 3),
            seed=self.seed_generator,
        )
        step_size = 1.0 / diffusion_steps
        for step in range(diffusion_steps):
            diffusion_times = ops.ones((num_images, 1, 1, 1)) - step * step_size
            noise_rates, signal_rates = diffusion_schedule(diffusion_times)
            pred_images, pred_noises = self.denoise(
                noisy_images, noise_rates, signal_rates
            )
```

`diffusion_steps` is a **generation-time** parameter — the same weights sample in 5 steps or 50.

SECTION 17.2.5

The generation loop: re-noise and finish

generate - re-noise and denormalize — PYTHON

```
            next_diffusion_times = diffusion_times - step_size
            next_noise_rates, next_signal_rates = diffusion_schedule(
                next_diffusion_times
            )
            noisy_images = (
                next_signal_rates * pred_images + next_noise_rates * pred_noises
            )

        images = (
            self.normalizer.mean + pred_images * self.normalizer.variance**0.5
        )
        return ops.clip(images, 0.0, 255.0)
```

The final two lines **undo the normalization** — multiply by the standard deviation, add the mean, clip to [0, 255]. The model works in normalized space throughout; only the very last step returns to pixels.

SECTION 17.2.6

There is no metric, so look at the pictures

PYTHON

```python
class VisualizationCallback(keras.callbacks.Callback):
    def __init__(self, diffusion_steps=20, num_rows=3, num_cols=6):
        self.diffusion_steps = diffusion_steps
        self.num_rows = num_rows
        self.num_cols = num_cols

    def on_epoch_end(self, epoch=None, logs=None):
        generated_images = self.model.generate(
            num_images=self.num_rows * self.num_cols,
            diffusion_steps=self.diffusion_steps,
        )
        for row in range(self.num_rows):
            for col in range(self.num_cols):
                i = row * self.num_cols + col
                plt.subplot(self.num_rows, self.num_cols, i + 1)
                img = ops.convert_to_numpy(generated_images[i]).astype("uint8")
                plt.imshow(img)
                plt.axis("off")
        plt.show()
```

This is chapter 7's callback API doing something it was not obviously designed for, and doing it well.

Two optimizer options that matter here

Learning rate decay

An `InverseTimeDecay` schedule gradually reduces the rate through training.

Exponential moving average

Also called **Polyak averaging**. Keep a running average of the weights, and every 100 batches overwrite the weights with it. Helps when the loss landscape is noisy.

PYTHON

```
model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=keras.optimizers.schedules.InverseTimeDecay(
            initial_learning_rate=1e-3, decay_steps=1000, decay_rate=0.1,
        ),
        use_ema=True,
        ema_overwrite_frequency=100,
    ),
)
```

Neither option changes what the model can represent. Both change **which minimum it settles into**, which for a generative model is the difference between plausible flowers and coloured smears.

Fit it, and do not forget to adapt the normalizer

PYTHON

```
model = DiffusionModel(image_size, widths=[32, 64, 96, 128], block_depth=2)
model.normalizer.adapt(dataset)

model.fit(
    dataset,
    epochs=100,
    callbacks=[
        VisualizationCallback(),
        keras.callbacks.ModelCheckpoint(
            filepath="diffusion_model.weights.h5",
            save_weights_only=True,
            save_best_only=True,
        ),
    ],
)
```

`model.normalizer.adapt(dataset)` computes the mean and variance the normalization needs. **Forget it and the noise and the images live on different scales**, and nothing works — with no error message.

100 epochs is about **90 minutes on a free Colab T4**.

Flowers that do not exist

After 100 epochs on 8,189 photographs, the model produces **convincing flowers**, and keeps improving if you keep training.

It is worth measuring what that took against what it produced: a U-Net of a few million parameters, a dataset small enough to download over coffee, and ninety minutes of a free GPU.

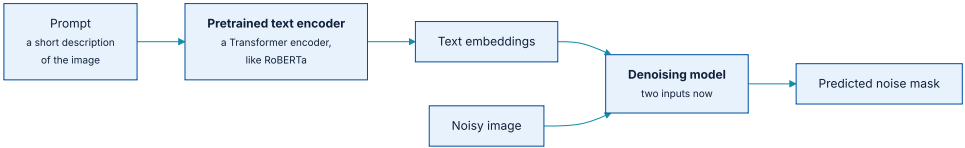
The next step to unlocking the real potential is **text conditioning** — which turns this into a model that produces images matching a given caption.

04

Text-to-image models

One extra input to the denoiser, and a walk through the space between two prompts.

Give the denoiser a second input



The denoising model now takes noisy_images and text_embeddings.

This gives a considerable **leg up** on the flower denoiser: instead of removing noise with no additional information, the model gets a textual representation of the final image to guide it.

Stable Diffusion in a few lines

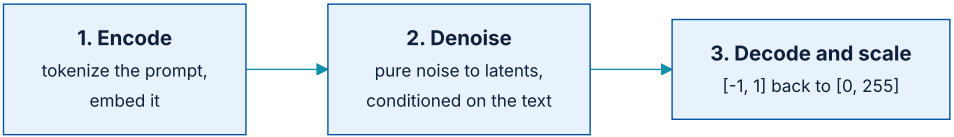
listing 17.8 PYTHON

```python
import keras_hub

height, width = 512, 512
task = keras_hub.models.TextToImage.from_preset(
    "stable_diffusion_3_medium",
    image_shape=(height, width, 3),
    dtype="float16",
)

prompt = "A NASA astronaut riding an origami elephant in New York City"
task.generate(prompt)
```

Like `CausalLM` last chapter, `TextToImage` is a **high-level task class** — it wraps tokenization and the whole diffusion process into one `generate()` call. `dtype="float16"` is a memory trick explained properly in chapter 18.

SECTION 17.3

What that one call is doing



Encode the prompt, denoise conditioned on it, rescale the output.

The middle stage is **exactly the flower model** — the same loop over a diffusion schedule, with one extra input to the denoiser. Everything the previous section built is still here.

Steering away from something

Train on **triplets** — `(image, positive_prompt, negative_prompt)`, where the positive describes the image and the negative is a series of words that do not. Feed both embeddings to the denoiser and it learns to move toward one and away from the other.

PYTHON

```
task.generate(
    {
        "prompts": prompt,
        "negative_prompts": "blue color",
    }
)
```

The result is the same scene with the blue removed — a control surface built entirely out of **how the training data was labelled**.

About those duplicated tusks

Unavoidable

Drawing a human in a space suit sitting on a paper elephant would require understanding **anatomy and physics** the model lacks. It interpolates from training data with no real understanding of the objects.

Easily fixable

We are using the **smallest** Stable Diffusion 3 release — about 3 billion parameters. A 9-billion-parameter version produces substantially fewer artifacts. It is omitted only to keep the example accessible.

Watching the denoising happen

PYTHON

```python
import numpy as np
from PIL import Image

def display(images):
    return Image.fromarray(np.concatenate(images, axis=1))

display([task.generate(prompt, num_steps=x) for x in [5, 10, 15, 20, 25]])
```

At 5 steps the image is a blurred impression; by 25 it is sharp. **The same weights throughout** — only the number of times they were applied changed.

Taking generate() apart

listing 17.9 PYTHON

```python
def get_text_embeddings(prompt):
    token_ids = task.preprocessor.generate_preprocess([prompt])
    negative_token_ids = task.preprocessor.generate_preprocess([""])
    return task.backbone.encode_text_step(token_ids, negative_token_ids)

def denoise_with_text_embeddings(embeddings, num_steps=28, guidance_scale=7.0):
    latents = random.normal((1, height // 8, width // 8, 16))
    for step in range(num_steps):
        latents = task.backbone.denoise_step(
            latents, embeddings, step, num_steps, guidance_scale,
        )
    return task.backbone.decode_step(latents)[0]
```

Note the latents are **1/8 the image size** per dimension: Stable Diffusion denoises in a compressed space and decodes to pixels only at the end.

The embedding is four tensors, not one

OUTPUT

```
>>> [x.shape for x in embeddings]
[(1, 154, 4096), (1, 154, 4096), (1, 2048), (1, 2048)]
```

Rather than passing only the final embedded vector, the Stable Diffusion authors pass **both** the final output vector and the last representation of the entire token sequence — more information for the denoiser to work with. Twice over, for positive and negative:

- The positive prompt's **encoder sequence** — (1, 154, 4096)
- The negative prompt's **encoder sequence** — (1, 154, 4096)
- The positive prompt's **encoder vector** — (1, 2048)
- The negative prompt's **encoder vector** — (1, 2048)

Interpolating on a sphere, not a line

listing 17.10 PYTHON

```python
def slerp(t, v1, v2):
    v1, v2 = ops.cast(v1, "float32"), ops.cast(v2, "float32")
    v1_norm = ops.linalg.norm(ops.ravel(v1))
    v2_norm = ops.linalg.norm(ops.ravel(v2))
    dot = ops.sum(v1 * v2 / (v1_norm * v2_norm))
    theta_0 = ops.arccos(dot)
    sin_theta_0 = ops.sin(theta_0)
    theta_t = theta_0 * t
    sin_theta_t = ops.sin(theta_t)
    s0 = ops.sin(theta_0 - theta_t) / sin_theta_0
    s1 = sin_theta_t / sin_theta_0
    return s0 * v1 + s1 * v2
```

The maths is not the point. **The motivation is.**

Why linear interpolation goes wrong

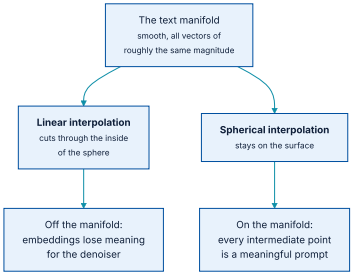


Figure 17.16 — spherical interpolation keeps us close to the surface of the manifold.

The manifold is of course not actually spherical. But it is a smooth surface of numbers all of roughly the same magnitude — it is **sphere-like**, and interpolating as if on a sphere is a better approximation than interpolating as if on a line.

SECTION 17.3.1

Interpolating the embeddings

PYTHON

```python
def interpolate_text_embeddings(e1, e2, start=0, stop=1, num=10):
    embeddings = []
    for t in np.linspace(start, stop, num):
        embeddings.append(
            (
                slerp(t, e1[0], e2[0]),
                e1[1],
                slerp(t, e1[2], e2[2]),
                e1[3],
            )
        )
    return embeddings
```

Elements 1 and 3 — the negative prompt's sequence and vector — are passed through unchanged from the first embedding.

A dog becomes something else entirely

PYTHON

```python
prompt1 = "A friendly dog looking up in a field of flowers"
prompt2 = ("A horrifying, tentacled creature hovering over a field of "
           "flowers")

e1 = get_text_embeddings(prompt1)
e2 = get_text_embeddings(prompt2)

images = []
for et in interpolate_text_embeddings(e1, e2, start=0.5, stop=0.6, num=9):
    image = denoise_with_text_embeddings(et)
    images.append(scale_output(image))
display(images)
```

The walk runs from **0.5 to 0.6** out of a full [0, 1] range — zoomed into the middle of the interpolation, right where the morph becomes visually obvious. Nine images across a **one-tenth** slice of the path.

Interpolation machines

> *This might feel like magic the first time you try it, but there's nothing magic about it —* ***interpolation is fundamental to the way deep neural networks learn.***
>
> Section 17.3.1

This is the last substantive model in the book, and a deliberately chosen visual metaphor to end on. Deep neural networks are **interpolation machines**: they map complex, real-world probability distributions onto low-dimensional manifolds.

We can exploit that fact even for input as complex as human language and output as complex as natural images — which is **exactly the claim chapter 15 made about attention**, arrived at from the other end.

COMMON FAILURE MODES

Four ways generative image work goes wrong

Forgetting normalizer.adapt()

The noise has unit variance; unnormalized images do not. Training runs, loss moves, and the output is **noise**. No error is raised.

Resizing without crop_to_aspect_ratio

Distortion in the training set becomes distortion in every generated image, and it is very hard to diagnose after the fact.

Linear interpolation of embeddings

Lands off the manifold. The midpoints decode to mush rather than to intermediate concepts. Use **slerp**.

Treating enhancement as evidence

Super-resolution invents detail from its priors. The moon-crater example is the standing warning: **never for forensics**.

SUMMARY

What has to survive this chapter

1. **Image generation means learning a latent space** that captures statistical information about a dataset — sample it, decode it, and you get images nobody has drawn.
2. **A classical autoencoder's latent space is not useful**; a VAE's is, because encoding to a distribution plus a KL term forces continuity.
3. **VAEs give structured, controllable latent spaces** — good for editing, concept vectors, and reconstruction, if not for fidelity.
4. **Override compute_loss(), not train_step()**, when leaving supervised learning — it keeps the code working on all three backends.

SUMMARY

What has to survive this chapter (2 of 2)

1. **Diffusion is a denoising autoencoder in a loop**, predicting the noise mask rather than the clean image.
2. **The schedule is the design**: diffusion time runs from 1 to 0, and a cosine relationship keeps signal^2 + noise^2 = 1 throughout.
3. **A U-Net with skip connections** is the denoiser — the same architecture as chapter 11's segmentation model, given a second scalar input.
4. **Text-to-image is one extra input** to the denoiser. Everything else is the flower model.
5. **Interpolation is what these models do.** Slerp between two prompts and you can watch it happen, frame by frame.

NOTEBOOK
05_stable_diffusion_latent_walk.ipynb

PAPER
Kingma & Welling, Auto-Encoding Variational Bayes

NEXT
Chapter 18 — Best practices for the real world